

**TRƯỜNG ĐẠI HỌC SÀI GÒN
KHOA CÔNG NGHỆ THÔNG TIN**



HỌC PHẦN:

**TRÍ TUỆ NHÂN TẠO NÂNG CAO
BÁO CÁO BÀI TẬP: SEARCHING**

GIẢNG VIÊN HƯỚNG DẪN: ĐỖ NHƯ TÀI

NHÓM SINH VIÊN THỰC HIỆN: ĐẶNG NHẬT ĐỨC – 3123410082

VÕ VĂN TUẤN - 3123410414

NIÊN KHOÁ: 2026 – 2027 / HỌC KỲ I

Thành phố Hồ Chí Minh, ngày 25 tháng 09 năm 2026

LỜI MỞ ĐẦU

Tìm kiếm đường đi trong mê cung là bài toán kinh điển đặt nền móng cho việc nghiên cứu không gian trạng thái và thuật toán tìm kiếm trong Trí tuệ Nhân tạo. Thông qua việc hình thức hóa bài toán thành các thành phần cốt lõi—bao gồm trạng thái đầu, trạng thái đích, không gian hành động, chi phí đường đi và cấu trúc cây tìm kiếm—báo cáo này tiến hành cài đặt và khảo sát toàn diện hai nhóm chiến lược chính: tìm kiếm không thông tin (BFS, DFS, Depth-Limited DFS, IDS) và tìm kiếm có thông tin (GBFS, A^* , Weighted A^*).

Bên cạnh việc phân tích vai trò định hướng của các hàm heuristic (Manhattan, Euclidean) cùng tầm quan trọng của cơ chế kiểm tra chu trình, báo cáo tập trung đánh giá định lượng hiệu năng thực nghiệm giữa các thuật toán dựa trên độ dài đường đi, số nút mở rộng, độ sâu nghiệm, kích thước tập biên (*frontier*) và thời gian thực thi. Qua đó, nghiên cứu làm rõ sự đánh đổi giữa mức độ tiêu hao tài nguyên bộ nhớ, chi phí tính toán và tính tối ưu của lời giải, cung cấp góc nhìn thực tiễn về việc lựa chọn thuật toán phù hợp cho các bài toán điều hướng không gian.

MỤC LỤC

LỜI MỞ ĐẦU	2
MỤC LỤC	3
NỘI DUNG	4
Bài 1. Cấu trúc cây tìm kiếm và lớp Node	4
1.1 Thuộc tính của một nút:	4
1.2 Vì sao cây tìm kiếm AI thường được tạo động và không nhất thiết là cây nhị phân:	4
Bài 2: Mô hình hóa bài toán mê cung	4
2.1 Định nghĩa bài toán tìm kiếm và xác định kích thước bài toán	4
2.2 Ước lượng kích thước bài toán	5
Bài 3. Tìm kiếm không thông tin: BFS và DFS	5
3.1 Thử nghiệm:	5
3.2 Sự khác nhau giữa Queue của BFS và Stack của DFS; liên hệ với tính đầy đủ, tối ưu và nhu cầu bộ nhớ.	7
3.3 Liên hệ với các đặc tính thuật toán	8
Bài 4: Depth-Limited DFS và Iterative Deepening Search (IDS)	8
Bài 5. Tìm kiếm có thông tin: GBFS và A*	9
5.1 So sánh GBFS và A*	9
5.2 Khảo sát A* với hai heuristic: khoảng cách Euclide và Manhattan.	10
Bài 6. Ảnh hưởng của heuristic	11
6.1 GBFS:	11
6.2 A*	12
6.3 Nhận xét và mở rộng	14
Bài 7. Kiểm tra chu trình trong DFS	15
7.1 Option_1: Kiểm tra trên đường đi:	15
7.2 Option_2: Kiểm tra trên đường đi + không thêm trạng thái đã có sẵn trong tập biên lần thứ hai.	15
7.3 Option_3: Kiểm tra đường đi và điều chỉnh frontier bằng cách đưa trạng thái tìm thấy lên đầu stack	16
7.4 So sánh số lần lặp, kích thước frontier và hiện tượng lặp trạng thái giữa ba phương án.	17
KẾT LUẬN CHUNG	19

NỘI DUNG

Bài 1. Cấu trúc cây tìm kiếm và lớp Node

1.1 Thuộc tính của một nút:

- pos (Trạng thái): Vị trí hoặc cấu hình cụ thể của bài toán tại thời điểm xét (dùng để nhận diện nút và kiểm tra xem đã đến đích chưa).
- parent (Nút cha): Tham chiếu trở về nút ngay trước nó, dùng để truy vết ngược lại toàn bộ đường đi khi tìm thấy đích.
- action (Hành động): Bước di chuyển cụ thể áp dụng từ nút cha để dẫn tới trạng thái hiện tại.
- cost (Chi phí tích lũy): Tổng chi phí tiêu tốn để đi từ nút gốc đến nút hiện tại (dùng làm cơ sở đánh giá trong UCS và A*).

1.2 Vì sao cây tìm kiếm AI thường được tạo động và không nhất thiết là cây nhị phân:

- Không gian trạng thái của các bài toán sử dụng tìm kiếm AI thường lớn và không biết trước. Vì vậy cây tìm kiếm sẽ thường được tạo dần trong quá trình thuật toán khám phá không gian trạng thái.
 - Trong AI, một trạng thái có thể có nhiều hơn 2 hành động hợp lệ để chuyển sang các trạng thái khác. Điều này yêu cầu cấu trúc của cây tìm kiếm phải thay đổi linh hoạt theo bài toán chứ không nhất thiết là cây nhị phân.
-

Bài 2: Mô hình hóa bài toán mê cung

2.1 Định nghĩa bài toán tìm kiếm và xác định kích thước bài toán

Xác định năm thành phần của bài toán tìm kiếm:

- Trạng thái đầu: Tọa độ của điểm S
- Tập hành động: Gồm bốn hành động mà tác tử có thể thực hiện bao gồm: E, N, S, W
- Mô hình chuyển trạng thái: Một trạng thái con được coi là hợp lệ và có thể cho phép tác tử chuyển qua nếu vị trí trạng thái con đó cũng thuộc không gian trạng

thái của bài toán (không phải là tường và cũng không nằm ngoài phạm vi mê cung).

- Trạng thái đích: Tọa độ của điểm G
- Chi phí đường đi: Là tổng số bước đã di chuyển tính từ vị trí khởi đầu.

2.2 Ước lượng kích thước bài toán

Bảng ước lượng các đại lượng của mê cung:

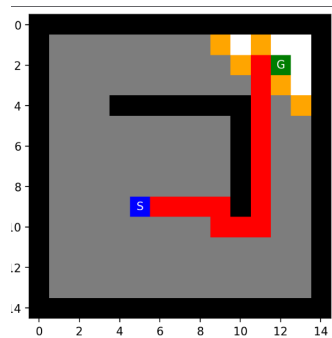
- n: kích thước không gian trạng thái
- d: độ sâu lời giải tối ưu
- m: độ sâu tối đa
- b: hệ số phân nhánh tối đa

Tên mê cung	n	d	m	b
L_maze.txt	157	$14 \leq d < 157$	156	4
empty_maze.txt	100	14	99	4
empty_maze_2.txt	100	14	99	4
large_maze.txt	647	$34 \leq d < 647$	646	4
loops_maze.txt	72	$9 \leq d < 72$	71	4
medium_maze.txt	274	$48 \leq d < 274$	273	4
open_maze.txt	684	$54 \leq d < 684$	683	4
small_maze.txt	94	$15 \leq d < 94$	93	4

Bài 3. Tìm kiếm không thông tin: BFS và DFS

3.1 Thử nghiệm:

BFS:



Path length: 16

Reached squares: 151

DFS:

Directions are checked in the order ['N', 'E', 'S', 'W']	Directions are checked in the order ['S', 'E', 'N', 'W']
Path length: 122 Reached squares: 145	Path length: 26 Reached squares: 53

Directions are checked at every step in random order.

(1)	(2)	(3)

Path length: 1550 Reached squares: 125	Path length: 482 Reached squares: 119	Path length: 188 Reached squares: 102
---	--	--

Nhận xét:

- Dựa vào kết quả trên ta có thể thấy BFS sẽ cho ta kết quả độ dài đường đi là ngắn hơn DFS rất nhiều.
- Số ô đã duyệt của BFS (Reached squares: 151) rất lớn, gần như lấp đầy không gian trống của mê cung. Điều này thể hiện đặc tính duyệt theo từng tầng/chiều rộng mở rộng dạng sóng tỏa ra xung quanh trước khi chạm tới đích.
- Đối với thuật toán DFS trong các trường hợp nếu ta bỏ đi thứ tự ưu tiên các hướng đi. Ta thấy rõ kết quả bắt đầu thiếu sự ổn định đến như thế nào. Số bước đi có thể lên đến hơn 1500 bước giải và gần đạt tới giới hạn độ dài của đường đi tối đa có thể đạt được. Điều này cho thấy việc chọn một thứ tự ưu tiên hành động phù hợp là rất quan trọng đối với thuật toán DFS
- Tuy nhiên chỉ từ kết quả trên mà ta cho rằng BFS tốt hơn DFS cũng không hẳn là chính xác. Nó còn phải phụ thuộc vào kích thước của mê cung, vị trí bắt đầu, vị trí kết thúc và cả cấu trúc tường chắn. Nếu cho một mê cung lớn hơn rất nhiều thì BFS cũng có thể tìm được lời giải ổn định nhưng không gian tìm kiếm của nó chưa chắc đã nhỏ DFS.

3.2 Sự khác nhau giữa Queue của BFS và Stack của DFS; liên hệ với tính đầy đủ, tối ưu và nhu cầu bộ nhớ.

- Cơ chế dữ liệu: FIFO vs. LIFO
 - Queue của BFS (First-In, First-Out):
 - Cơ chế: Đỉnh nào được phát hiện trước sẽ được lấy ra xét trước.
 - Hành vi duyệt: Lan tỏa đều theo từng tầng (breadth-first). Thuật toán khám phá cạn kiệt toàn bộ các nút ở độ sâu d trước khi chạm đến bất kỳ nút nào ở độ sâu $d + 1$.
- Stack của DFS (Last-In, First-Out):
 - Cơ chế: Đỉnh nào được thêm vào sau cùng (nhánh con sâu nhất vừa sinh ra) sẽ được ưu tiên lấy ra xét trước.
 - Hành vi duyệt: Lao thẳng xuống đáy của một nhánh (depth-first). Thuật toán đi sâu hết mức có thể dọc theo một đường đi cho đến khi gặp ngõ cụt mới quay lui (backtrack).

3.3 Liên hệ với các đặc tính thuật toán

Tính đầy đủ (Completeness)

- BFS (Đầy đủ): Do Queue ép thuật toán duyệt tuần tự theo từng mức độ sâu $0, 1, 2, \dots$, BFS chắc chắn sẽ tìm thấy nghiệm nếu nghiệm tồn tại ở độ sâu hữu hạn d , ngay cả khi cây tìm kiếm có các nhánh vô tận ($m = \infty$) với $b < \infty$.
- DFS (Không đầy đủ trong đồ thị vô hạn hoặc có chu trình): Do Stack liên tục đẩy các nút con vào đỉnh ngăn xếp, DFS có nguy cơ rơi vào nhánh vô tận hoặc vòng lặp vô hạn (nếu không kiểm tra trạng thái lặp - graph search), khiến thuật toán không bao giờ quay lui để tìm thấy nghiệm ở nhánh khác.

Tính tối ưu (Optimality)

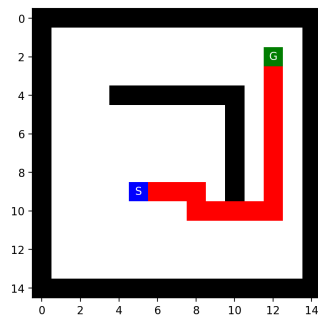
- BFS (Tối ưu về số bước đi / chi phí bằng nhau): Khi chi phí của mỗi bước đi là như nhau (uniform step cost), đường đi đầu tiên mà BFS chạm tới nghiệm được đảm bảo có độ sâu d nhỏ nhất, tức là tối ưu.
- DFS (Không tối ưu): Stack ép thuật toán chọn đi sâu vào nhánh đầu tiên duyệt trúng. Do đó, DFS sẽ trả về nghiệm đầu tiên mà nó gặp ở dưới đáy nhánh đó, ngay cả khi nghiệm đó nằm ở độ sâu $m \gg d$ trong khi một nghiệm ngắn hơn nằm ngay nhánh bên cạnh.

Nhu cầu bộ nhớ (Space Complexity)

- BFS (Nhu cầu bộ nhớ cực lớn - Điểm yếu chí mạng):
 - Độ phức tạp: $O(b^d)$.
 - Lý do: Queue phải lưu trữ đồng thời toàn bộ biên của tầng hiện tại để mở rộng tầng kế tiếp. Ở tầng d , số lượng nút chờ trong Queue xấp xỉ b^d . Bộ nhớ RAM thường cạn kiệt rất lâu trước khi thuật toán kịp hoàn tất duyệt.
 - DFS (Nhu cầu bộ nhớ rất tiết kiệm - Điểm mạnh vượt trội):
 - Độ phức tạp: $O(b \cdot m)$ (hoặc $O(m)$ nếu chỉ lưu vết đường đi hiện tại).
 - Lý do: Stack chỉ cần lưu vết đường đi từ gốc đến nút lá hiện tại cùng với các nút anh em chưa xét dọc theo đường đi đó. Khi một nhánh kết thúc, các nút được lấy ra khỏi Stack và giải phóng bộ nhớ.
-

Bài 4: Depth-Limited DFS và Iterative Deepening Search (IDS)

Kết quả chạy IDS với cách quản lý bộ nhớ của DFS:



Nhận xét: Đường đi vô cùng tối ưu, tương tự BFS nhưng lại không có vị trí thừa nào được lưu giữ lại trong bộ nhớ sau khi tìm được goal.

Vì sao IDS kết hợp đặc tính bộ nhớ của DFS với khả năng tìm lời giải theo độ sâu tăng dần?

Nếu chỉ sử dụng DFS thuần túy, trong những bài toán có không gian trạng thái rất lớn thậm chí không có biên giới hạn thì DFS rất dễ dẫn tác tử đi ra xa khỏi vị trí goal và không bao giờ quay đầu lại. Tuy nhiên đặc tính bộ nhớ nhỏ gọn, không lưu phần tử thừa của DFS lại là một lợi thế trong những bài toán lớn.

Bài 5. Tìm kiếm có thông tin: GBFS và A*

5.1 So sánh GBFS và A*

GBFS:	A*:

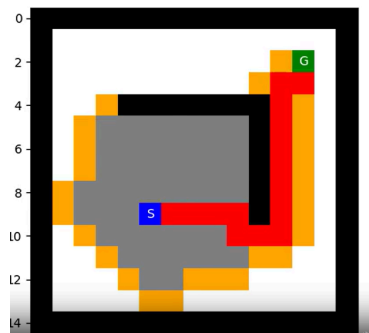
Directions are checked at every step in random order. CPU times: user 1.21 ms, sys: 119 μs, total: 1.33 ms Wall time: 1.33 ms Path length: 22 Reached squares: 61	Directions are checked at every step in random order. CPU times: user 1.49 ms, sys: 144 μs, total: 1.63 ms Wall time: 1.63 ms Path length: 16 Reached squares: 67
---	---

Nhận xét:

- Tính tối ưu đường đi: A* vượt trội hơn khi tìm được đường đi ngắn nhất (16 bước so với 22 bước của GBFS). GBFS chỉ dựa vào $h(n)$ nên bị chướng ngại vật đánh lừa đi vòng lên trên rồi mới quay lại.
- Không gian tìm kiếm: GBFS duyệt ít ô hơn (61 ô so với 67 ô của A*) do bản chất tham lam chỉ tập trung tiến về đích; ngược lại, A* phải duyệt thêm một số nhánh lân cận để bảo đảm tính tối ưu chi phí.
- Thời gian thực thi: GBFS chạy nhanh hơn (1.33 ms so với 1.63 ms của A*) nhờ duyệt ít trạng thái hơn và hàm đánh giá tính toán đơn giản hơn.
- Kết luận: GBFS ưu tiên tốc độ nhưng dễ chọn đường đi kém tối ưu; A* đánh đổi thêm một chút thời gian và số ô duyệt để đảm bảo tìm ra đường đi ngắn nhất.

5.2 Khảo sát A* với hai heuristic: khoảng cách Euclide và Manhattan.

A* với Euclide:



CPU times: user 8.6 ms, sys: 44 μ s, total: 8.65 ms

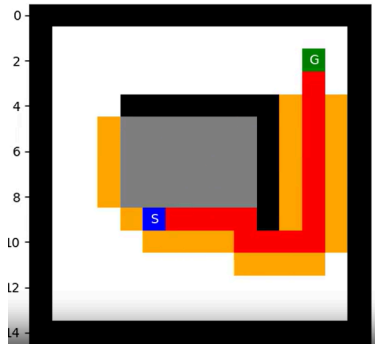
Wall time: 8.32 ms

Path length: 16

Reached squares: 87

Action sequence: ['E', 'E', 'E', 'E', 'S', 'E', 'E', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'E', 'N']

A* với Manhattan



CPU times: user 7.4 ms, sys: 0 ns, total: 7.4 ms

Wall time: 7.19 ms

Path length: 16

Reached squares: 66

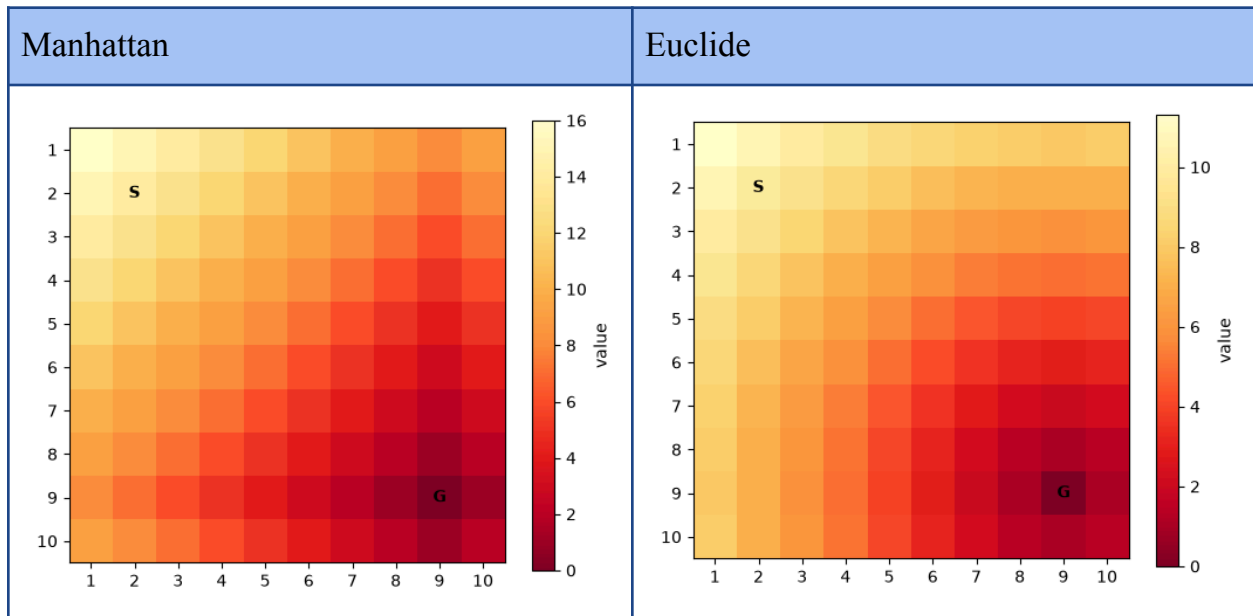
Action sequence: ['E', 'E', 'E', 'E', 'S', 'E', 'E', 'E', 'N', 'N', 'N', 'N', 'N', 'N', 'N', 'N']

Nhận xét:

- Ghi nhận nhận xét của notebook: GBFS và A* đều tìm đường ngắn nhất trong thiết lập này; A* có thể mở rộng ít trạng thái hơn đáng kể nếu heuristic phù hợp.
 - Cả thuật toán GBFS và A* đều mang tính tối ưu và tìm ra đường đi ngắn nhất có thể trong tình huống này. GBFS duyệt qua 61 trạng thái so sánh với 67 trạng thái của A* thì không khác nhau mấy. Về không gian cần lưu trữ một cây tìm kiếm của hai thuật toán cũng có kích thước tương đương
 - Hiệu quả của A* phụ thuộc vào chất lượng của hàm heuristic. Khoảng cách Manhattan là một hàm heuristic rất tốt cho bài toán này.
 - Đối với một số bài toán AI, cây tìm kiếm thu nhỏ mà A* tạo ra vẫn có thể vượt quá dung lượng bộ nhớ khả dụng. Trong trường hợp đó, các phương pháp khác như tìm kiếm A* có trọng số (weighted A*) hoặc tìm kiếm sâu dần (iterative deepening search - IDS) có thể cần được sử dụng.
-

Bài 6. Ảnh hưởng của heuristic

6.1 GBFS:



Đặc điểm phân bố giá trị:

- Manhattan: Giá trị giảm đều tạo thành các dải màu chéo góc 45° . Rất nhiều ô lân cận có giá trị bằng nhau.
- Euclidean: Giá trị giảm theo các cung tròn đồng tâm hướng về đích G . Các ô lân cận hầu như luôn có giá trị khác nhau.

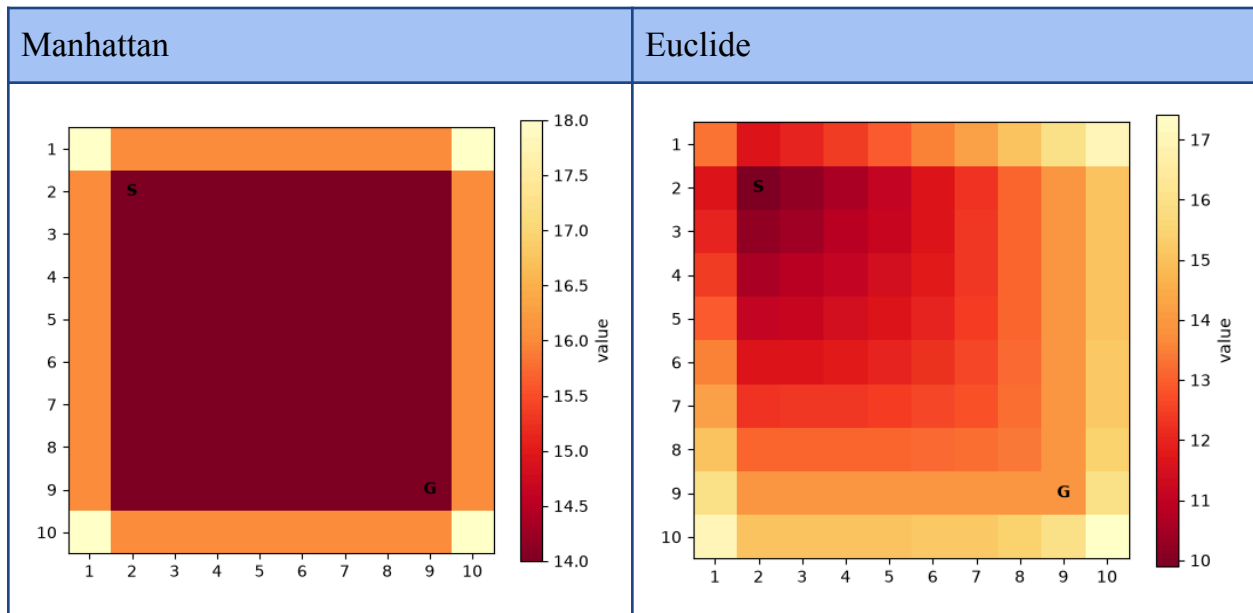
Hình dạng đường đi tạo ra:

- Manhattan: Do liên tục bị bằng điểm (tie), đường đi phụ thuộc hoàn toàn vào thứ tự ưu tiên các hướng trong code (ví dụ: luôn đi Xuống trước khi đi Sang Phải). Kết quả là tạo ra đường đi gấp góc chữ L (chạy kịch một trục rồi mới rẽ sang trục kia).
- Euclidean: Do đo khoảng cách đường chim bay, thuật toán tự động chọn các bước luân phiên (xuống - phải - xuống - phải) để bám sát đường chéo nối thẳng từ S đến G .

Hiệu năng tìm kiếm:

- Cả hai đều tìm đến đích rất nhanh chỉ sau 15 bước mở rộng (không bị quay lui vì không có tường chắn).
- Euclid định hướng hình học tự nhiên hơn, trong khi Manhattan phụ thuộc chặt vào quy tắc hòa điểm (tie-breaking).

6.2 A*



Đặc điểm phân bố giá trị (f -value landscape):

- Manhattan:
 - Mọi ô nằm trong hình chữ nhật bao quanh bởi S và G (từ hàng 2 đến 9, cột 2 đến 9) đều có cùng giá trị f thấp nhất (bằng 14).
 - Khu vực này tạo thành một "vùng lòng chảo bằng phẳng" màu đỏ thẫm lớn chiếm trọn không gian giữa điểm đầu và điểm cuối.
- Euclidean:
 - Giá trị f không bằng phẳng mà tăng dần khi đi xa dần khỏi điểm xuất phát S (thấp nhất tại S với giá trị xấp xỉ 10 và tăng dần về phía G).
 - Gradient màu có dạng các góc bo tròn lan tỏa, phản ánh việc hàm Euclid ước lượng chi phí chặt chẽ khác nhau ở từng tọa độ.

Hành vi tìm kiếm và mở rộng (Search Behavior):

- Manhattan:
 - Vì toàn bộ vùng ô vuông giữa S và G đều có chung giá trị $f = 14$ nhỏ nhất, A* rơi vào tình trạng đồng điểm (tie) cực lớn.
 - Nếu không có cơ chế phá vỡ hòa điểm tốt (ví dụ: ưu tiên h nhỏ hơn khi f bằng nhau), A* có xu hướng mở rộng lan man ra rất nhiều ô trong vùng phẳng này trước khi chạm đích.
- Euclidean:
 - Giá trị h của Euclidean nhỏ hơn chi phí thực tế trên lưới 4 hướng, khiến giá trị f phân tầng rõ rệt.

- Hàng đợi ưu tiên có sự chênh lệch rõ ràng giữa các ô, giảm hẳn việc bằng điểm, giúp luồng duyệt tập trung và có định hướng gradient cụ thể hơn.

Hiệu năng và độ tối ưu:

- Cả hai đều đảm bảo tìm được đường đi tối ưu ngắn nhất (độ dài bằng 14 bước).
- Manhattan cho ước lượng chi phí sát với thực tế lưới 4 hướng hơn (chi phí thực tế chính xác là 14), nhưng lại tạo ra vùng bằng phẳng lớn khiến việc định hướng phụ thuộc chặt chẽ vào quy tắc hòa điểm.
- Euclid phân biệt tốt thứ tự giữa các trạng thái, song do ước lượng non hơn (underestimate nhiều hơn), không gian tìm kiếm có xu hướng lan rộng theo hình elip quanh đường nối $S - G$.

6.3 Nhận xét và mở rộng

Vì sao Manhattan thường phù hợp với bài toán di chuyển 4 hướng trên lưới hơn Euclid:

- Phản ánh đúng không gian hành động: Khi tác nhân chỉ được di chuyển 4 hướng (trên, dưới, trái, phải), khoảng cách thực tế giữa hai ô luôn bằng tổng độ lệch theo hai trục ngang và dọc, đúng bằng cách tính của khoảng cách Manhattan.
- Ước lượng sát thực tế hơn (Tighter Heuristic): Euclid đo khoảng cách đường chim bay (cho phép đi chéo), dẫn đến việc đánh giá chi phí bị non hơn đáng kể (underestimate quá nhiều). Manhattan cho giá trị sát với chi phí thực tế nhất có thể mà vẫn đảm bảo tính chấp nhận được (admissible), giúp thuật toán A^* thu hẹp không gian tìm kiếm và mở rộng ít trạng thái thừa hơn.

Đề xuất heuristic khác cho A^* :

Manhattan có phá vỡ đồng điểm (Tie-breaking Manhattan):

- Công thức dạng: $h'(n) = h_M(n) \times (1 + \epsilon)$ (với ϵ rất nhỏ, cỡ 0.001).
- Giữ nguyên độ chính xác của Manhattan nhưng triệt tiêu hiện tượng bằng điểm, giúp A^* đi thẳng đến đích mà không duyệt lan man.

Điều kiện bảo đảm tính tối ưu của A^*

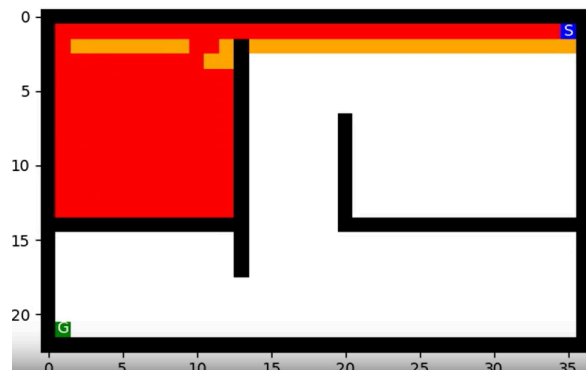
- Tính chấp nhận được (Admissibility) — cho Tree Search: Heuristic không được ước lượng vượt quá chi phí thực tế ($h(n) \leq h^*(n)$ và $h(Goal) = 0$).
 - Tính nhất quán (Consistency) — cho Graph Search: Thỏa mãn bất đẳng thức tam giác ($h(n) \leq c(n, n') + h(n')$), đảm bảo giá trị f không giảm và không phải duyệt lại các nút đã đóng.
-

Bài 7. Kiểm tra chu trình trong DFS

7.1 Option_1: Kiểm tra trên đường đi:

Không mở rộng các nút đại diện cho trạng thái đã xuất hiện trên đường đi hiện tại. Cách hoạt động như sau:

- Quá trình mở rộng (Expand) đưa tất cả các trạng thái mới vào tập biên (frontier).
- Khi một trạng thái được lấy ra khỏi tập biên, chúng ta sẽ kiểm tra xem đã có nút nào mang trạng thái này trên đường đi hiện tại tính từ nút gốc hay chưa. Nếu đã có, chúng ta bỏ qua trạng thái đó và chọn trạng thái tiếp theo từ tập biên. Khi cố gắng thêm một nút từ tập biên vào đường đi hiện tại, chúng ta kiểm tra xem trạng thái đó đã tồn tại trên đường đi hay chưa.



(DFS-Option_1)

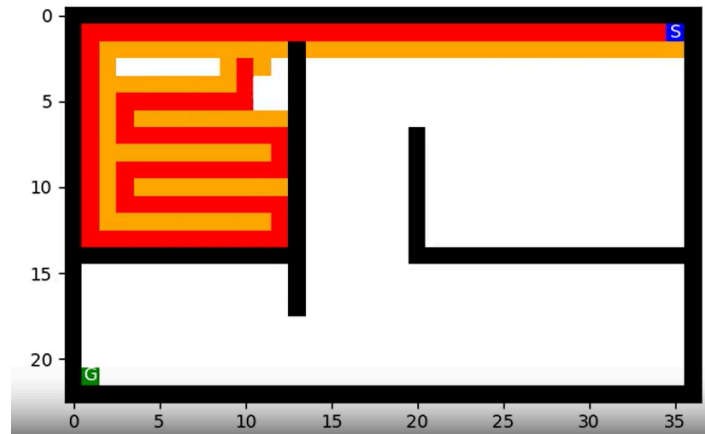
Cách tiếp cận này bộc lộ nhược điểm là quản lý tập biên không tốt, bởi vì có những trường hợp cùng một trạng thái bị thêm nhiều lần vào tập biên, dẫn đến việc phải giải phóng không gian bộ nhớ rồi liên tục duyệt lại trạng thái đó nhiều lần.

7.2 Option_2: Kiểm tra trên đường đi + không thêm trạng thái đã có sẵn trong tập biên lần thứ hai.

Nhằm khắc phục việc tập biên phình to ở mục 7.1, phương pháp này bổ sung thêm bước lọc trạng thái trùng ngay lúc sinh ra nút mới:

- Khi mở rộng một nút, chỉ đưa trạng thái con vào tập biên (frontier) nếu trạng thái đó chưa xuất hiện trên đường đi hiện tại và chưa tồn tại sẵn trong tập biên.
- Khi lấy một nút ra khỏi frontier, nút đó được đảm bảo là duy nhất ở thời điểm thêm vào; ta chỉ cần kiểm tra nhanh tính hợp lệ với đường đi hiện tại trước khi tiếp tục mở rộng.

- Ưu điểm & Hạn chế: Tiết kiệm đáng kể bộ nhớ và tránh việc duyệt lại một trạng thái nhiều lần. Đổi lại, thuật toán cần duy trì một cấu trúc dữ liệu hỗ trợ (như bảng băm / hash set) để kiểm tra sự tồn tại trong frontier với chi phí $O(1)$.

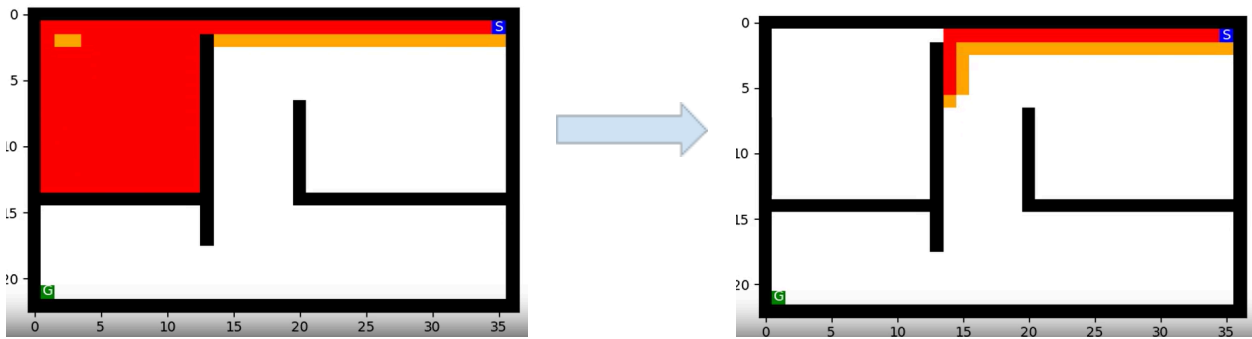


(DFS-Option_2.)

7.3 Option_3: Kiểm tra đường đi và điều chỉnh frontier bằng cách đưa trạng thái tìm thấy lên đầu stack

Phương pháp này tối ưu hóa tính chất LIFO (vào sau, ra trước) của DFS khi phát hiện một trạng thái đã tồn tại trong tập biên:

- Khi mở rộng, nếu một trạng thái con đã có sẵn trong tập biên (frontier) nhưng chưa nằm trên đường đi hiện tại, thuật toán không bỏ qua mà sẽ di chuyển trạng thái đó lên đỉnh ngăn xếp (hoặc xóa vị trí cũ và đẩy vị trí mới vào đầu stack).
- Việc này giúp DFS ưu tiên đào sâu ngay vào nhánh tìm kiếm vừa tìm thấy trạng thái đó, phản ánh hành vi bám sát độ sâu mới nhất thay vì để trạng thái bị chìm sâu dưới đáy stack.
- Ưu điểm & Hạn chế: Tránh việc lặp lại trạng thái trong frontier, đồng thời khai thác tốt hơn tính cục bộ của DFS. Tuy nhiên, thao tác tìm kiếm và điều chỉnh vị trí phần tử bên trong stack đòi hỏi cấu trúc dữ liệu phức tạp hơn để tránh suy giảm hiệu năng.



(DFS-Option_3)

7.4 So sánh số lần lặp, kích thước frontier và hiện tượng lặp trạng thái giữa ba phương án.

Bảng so sánh chi tiết giữa 3 phương án kiểm tra chu trình trong DFS:

Tiêu chí	7.1 Kiểm tra trên đường đi (Check the path)	7.2 Kiểm tra đường đi + Không thêm trùng vào frontier	7.3 Kiểm tra đường đi + Đưa trạng thái lên đầu stack
Kích thước Frontier	Lớn nhất: Có thể phình to đáng kể do chứa nhiều nút mạng cùng một trạng thái được sinh ra từ các nhánh khác nhau.	Nhỏ gọn: Giới hạn nghiêm ngặt, mỗi trạng thái chỉ xuất hiện tối đa một lần trong frontier tại một thời điểm.	Nhỏ gọn tương đương 7.2: Luôn duy trì tính duy nhất của mỗi trạng thái trong frontier (không có phần tử trùng lặp).
Hiện tượng lặp trạng thái	Nghiêm trọng: Cùng một trạng thái có thể bị đẩy vào frontier nhiều lần, dẫn đến việc lấy ra, kiểm tra rồi bỏ qua hoặc duyệt lặp lại nhiều lần.	Bị triệt tiêu hoàn toàn trong frontier: Trạng thái đã nằm trong frontier sẽ không bao giờ bị thêm lần thứ hai.	Bị triệt tiêu hoàn toàn: Không tạo bản sao; thay vào đó chỉ cập nhật lại vị trí/độ ưu tiên của trạng thái đã có.
Số lần lặp (Số bước duyệt vòng lặp)	Nhiều nhất: Tồn thêm rất nhiều lượt lặp của vòng lặp chính chỉ để pop các nút trùng lặp ra khỏi frontier và loại bỏ chúng.	Ít hơn đáng kể so với 7.1: Loại bỏ hoàn toàn các lượt lặp xử lý nút rác/nút trùng lấy ra từ frontier.	Tương đương hoặc ít hơn 7.2: Bằng việc đẩy trạng thái lên đầu stack, thuật toán tập trung đào sâu ngay nhánh mới nhất, có thể tiếp cận đích nhanh hơn tùy dạng cây tìm kiếm.

Chi phí quản lý cấu trúc	Thấp nhất: Chỉ dùng stack thông thường và một tập hợp kiểm tra các nút trên đường đi hiện tại.	Trung bình: Cần kết hợp thêm bảng băm (hash set) để tra cứu nhanh sự tồn tại của trạng thái trong frontier với $O(1)$.	Cao nhất: Cần cấu trúc dữ liệu kết hợp (như Dictionary + Doubly Linked List) để vừa tra cứu vừa xóa/chuyển nút lên đỉnh stack với chi phí $O(1)$.
---------------------------------	--	--	--

KẾT LUẬN CHUNG

1. Về tính Đầy đủ (Completeness) và Tính Tối ưu (Optimality)

• Tìm kiếm không thông tin (Uninformed Search):

- BFS đảm bảo cả tính đầy đủ và tính tối ưu (trong không gian có chi phí bước đi bằng nhau/uniform cost). BFS tìm nghiệm ngắn nhất nhờ khám phá không gian theo dạng sóng lan tỏa đồng tâm.
- DFS không đảm bảo tính tối ưu vì bản chất ưu tiên lao sâu vào nhánh tìm kiếm đầu tiên nó gặp, dễ bị dẫn dắt vào các đường vòng vèo có độ dài lớn gấp nhiều lần nghiệm tối ưu. Hơn nữa, DFS chỉ bảo đảm tính đầy đủ trong không gian hữu hạn có cơ chế kiểm tra trạng thái lặp chặt chẽ.
- IDS (Iterative Deepening Search) là giải pháp dung hòa xuất sắc: vừa đạt tính đầy đủ và nghiệm tối ưu như BFS (do tăng dần giới hạn độ sâu từng bước), vừa kế thừa cơ chế tiết kiệm tài nguyên của DFS.

• Tìm kiếm có thông tin (Informed Search):

- GBFS chỉ đánh giá dựa trên $h(n)$, bản chất tham lam cục bộ khiến thuật toán không đảm bảo tối ưu và rất dễ bị “đánh lừa” bởi chướng ngại vật dạng hốc chữ L, chữ U.
- A* kết hợp giữa chi phí đã đi $g(n)$ và ước lượng tương lai $h(n)$ ($f(n) = g(n) + h(n)$). Khi hàm heuristic thỏa mãn tính chấp nhận được (admissible) đối với cây tìm kiếm hoặc tính nhất quán (consistent) đối với đồ thị, A* luôn bảo đảm tìm ra đường đi tối ưu tuyệt đối.

2. Sự đánh đổi giữa Thời gian thực thi và Dung lượng bộ nhớ (Time & Space Trade-offs)

• Bộ nhớ (Space Complexity) – Thách thức lớn nhất:

- Điểm yếu chí mạng của BFS là tập biên (Queue) phình to theo cấp số nhân $O(b^d)$. Trong các mê cung lớn, BFS thường làm cạn kiệt RAM hệ thống trước khi chạm tới nghiệm.
- Ngược lại, DFS và IDS chỉ tiêu tốn bộ nhớ tuyến tính $O(bm)$ hoặc $O(bd)$, giải phóng ngay các nút sau khi duyệt xong nhánh, giúp thuật toán mở rộng được trong các không gian trạng thái cực lớn.

- **A* và GBFS:** Nhờ có sự dẫn đường của thông tin heuristic, A* và GBFS định hướng trực tiếp về phía đích, thu hẹp đáng kể số nút cần duyệt so với BFS. Tuy nhiên, A* trên đồ thị (Graph Search) vẫn phải lưu trữ cả tập mở (Open) và tập đóng (Closed), do đó vẫn phải chịu giới hạn về bộ nhớ trong các bài toán quy mô lớn.

3. Vai trò và Ảnh hưởng của Hàm Heuristic

- **Tính chất hình học của không gian hành động:**
 - Khi chuyển động giới hạn trong lưới 4 hướng (N, E, S, W), khoảng cách Manhattan phản ánh chính xác cấu trúc bài toán hơn Euclid. Khoảng cách Euclid xem xét đường chéo (chim bay), dẫn tới việc ước lượng non hơn (underestimate quá nhiều), làm A* phải mở rộng một vùng elip trạng thái rộng hơn quanh trục xuất phát - đích.
- **Vấn đề đồng điểm (Tie-breaking):**
 - Thực nghiệm biểu diễn cảnh quan hàm $f(n)$ (f-value landscape) cho thấy Manhattan tạo ra một “vùng lòng chảo bằng phẳng” rộng lớn với cùng giá trị f tối ưu giữa S và G . Điều này làm thuật toán mất phương hướng cục bộ nếu không có quy tắc phá vỡ đồng điểm (tie-breaking).
 - Việc hiệu chỉnh heuristic, chẳng hạn bổ sung hệ số nhỏ $h'(n) = h(n) \times (1 + \epsilon)$ hoặc ưu tiên nút có $h(n)$ nhỏ hơn khi $f(n)$ bằng nhau, đóng vai trò quyết định giúp A* đi thẳng một mạch đến đích mà không bị tản mát.

4. Tầm quan trọng của Cơ chế Kiểm tra Chu trình (Cycle Checking) trong DFS

Khảo sát 3 phương án quản lý chu trình trong DFS khẳng định:

- **Kiểm tra chỉ trên đường đi hiện tại (Option 1)** khiến tập biên bị phình to nghiêm trọng do nhận nhiều trạng thái lặp sinh ra từ các nhánh song song, dẫn đến tiêu tốn chu kỳ CPU để pop và loại bỏ nút rác.
- **Kiểm tra đường đi + không thêm trạng thái đã có sẵn trong tập biên (Option 2)** giúp thu nhỏ triệtm để kích thước Frontier, triệt tiêu lặp trạng thái với chi phí duy trì bảng băm $O(1)$ rất hiệu quả.
- **Đưa trạng thái tìm thấy lên đỉnh Stack (Option 3)** khai thác tối đa tính cục bộ và hành vi LIFO của DFS, giúp thuật toán luôn bám sát hướng rẽ mới nhất, giảm số bước duyệt dư thừa nhưng đòi hỏi cấu trúc dữ liệu kết hợp phức tạp hơn.

Tổng kết

Không có một thuật toán đơn lẻ nào tối ưu trong mọi tình huống:

- Khi không gian trạng thái nhỏ và yêu cầu nghiệm ngắn nhất tuyệt đối, BFS hoặc A* là lựa chọn hàng đầu.
- Khi bộ nhớ bị giới hạn nghiêm ngặt, IDS là giải pháp thay thế tối ưu cho BFS.
- Trong các bài toán điều hướng và quy hoạch đường đi thực tế, A* kết hợp hàm heuristic Manhattan có cơ chế tie-breaking (hoặc Weighted A* khi muốn đánh đổi một phần độ tối ưu để đạt tốc độ giải quyết nhanh vượt trội) mang lại hiệu quả cân bằng nhất giữa chi phí tính toán và chất lượng lời giải.